Linguaggi di Programmazione

Parser Top-down e grammatiche LL(k)

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Parser Top-down

Parser Top-down Nondeterministico

Data $G = (NT, T, S, R)$ libera, costruiamo il PDA con un solo stato $M = (T, \{q\}, T \cup NT, \delta, q, S, \emptyset)$ che riconosce per pila vuota, dove δ è definita come segue:

- $(q, \beta) \in \delta(q, \epsilon, A)$ se $A \to \beta \in R$ (espandi)
- $(q, \epsilon) \in \delta(q, a, a)$ per ogni $a \in T$ (consuma)

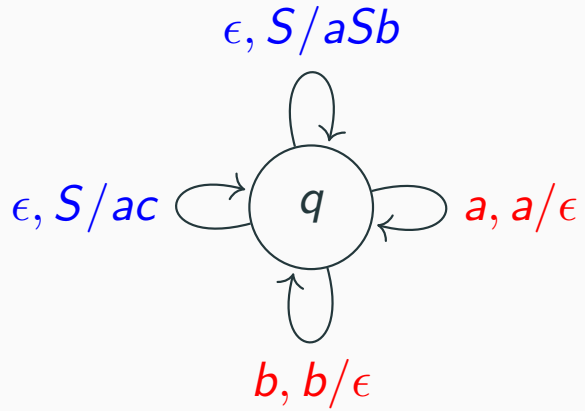
tale che $L(G) = P[M]$.

N.B.: le transizioni di tipo espandi generano nondeterminismo!

Questo tipo di automa è un basico parser top-down nondeterministico.

Esempio

Data la grammatica $G = S \to aSb \mid ac$, il suo PDA top-down N è:



Consideriamo la seguente computazione del PDA N con input $aacb$:

$(q, aacb, S) \vdash_N (q, aacb, aSb) \vdash_N (q, acb, Sb) \vdash_N (q, acb, acb) \vdash_N$
$(q, cb, cb) \vdash_N (q, b, b) \vdash_N (q, \epsilon, \epsilon)$

Ogni passo di tipo espandi produce un passo della derivazione **leftmost**, dalla quale si ricava un albero di derivazione. La derivazione leftmost è $S \Rightarrow aSb \Rightarrow aacb$.

Nondeterminismo

- Questo parser top-down è molto inefficiente a causa del **nondeterminismo**: se la produzione scelta è sbagliata, bisogna fare backtracking. Quindi, nel caso peggiore, c'è la necessità di esplorare tutte le alternative!
- Nel caso pessimo, la complessità è esponenziale nella lunghezza della stringa w, dove la base b è data dal massimo numero di produzioni per uno stesso nonterminale: $O(b^{|w|})$.

Cerchiamo di guidare la scelta della produzione. Come? Guardando il prossimo carattere (o i prossimo caratteri) dell'input da leggere (look-ahead). Ad esempio:

- $A \to aB \mid bC$
  - se il prossimo input è a, uso $A \to aB$
  - se il prossimo input è b, uso $A \to bC$
- $S \to ac \mid aSb$
  - se i prossimi 2 input sono ac, uso $S \to ac$
  - se i prossimi 2 input sono aa, uso $S \to aSb$

Attenzione!

- Da qui in avanti assumeremo di avere un simbolo speciale \$, che non faccia parte dei simboli di nessuna grammatica, che useremo come segnalatore di fine input (end marker). Cioè, invece di considerare un linguaggio L, consideriamo il linguaggio $L\$ = \{w\$ \mid w \in L\}$.
- Perchè? Un parser top-down deterministico è essenzialmente un DPDA che riconosce per pila vuota. Allora è necessario che il linguaggio sia prefix-free, e $L\$$ sicuramente gode di questa proprietà.

Data una produzione $A \to \alpha$, per determinare quali siano i simboli terminali che α può inizialmente generare, introduciamo una funzione ausiliaria: **First**.

Poi vedremo che sarà necessaria anche un'altra funzione, detta **Follow**, che determina quali simboli terminali possano seguire un nonterminale, per poter gestire adeguatamente le ϵ-produzioni (e i simboli annullabili).

First e Follow

First

Data una grammatica libera $G = (NT, T, S, R)$ e $\alpha \in (T \cup NT)^*$, diciamo che $First(\alpha)$ è l'insieme dei **terminali** che possono stare in prima posizione di una stringa che si deriva da α.

- per $a \in T$, $a \in First(\alpha)$ sse $\alpha \Rightarrow^* a\beta$ per $\beta \in (T \cup NT)^*$
- inoltre, se $\alpha \Rightarrow^* \epsilon$, allora $\epsilon \in First(\alpha)$

Quindi: $First : (T \cup NT)^* \to \mathcal{P}(T \cup \{\epsilon\})$, cioè $First(\alpha) \subseteq T \cup \{\epsilon\}$.

N.B.: Prendiamo $A \to \alpha_1 \mid \alpha_2$
Se $First(\alpha_1) \cap First(\alpha_2) = \emptyset$, allora la scelta della produzione è deterministica.
Ad esempio, sia $A \to aB \mid bC$: $First(aB) = \{a\}$ e $First(bC) = \{b\}$, quindi ho determinismo con un solo carattere di look-ahead.

Ma il First può non bastare ....

Consideriamo la seguente grammatica:

$$S \to Ab \mid c \qquad\qquad A \to aA \mid \epsilon$$

Ho che

$First(Ab) = \{a, b\}$ $\quad$ b perché $A \to \epsilon$

$First(c) = \{c\}$

$First(Ab) \cap First(c) = \emptyset$

$First(aA) = \{a\}$

$First(\epsilon) = \{\epsilon\}$

$First(Aa) \cap First(\epsilon) = \emptyset$

Però, se l'input è ab, abbiamo $(q, ab, S) \vdash_N (q, ab, Ab) \vdash_N (q, ab, aAb) \vdash_N (q, b, Ab)$

ma ora non so quale produzione per A scegliere, perché b non compare nei First di nessuna delle due produzioni per A!

Devo anche valutare cosa può seguire A, perché A può annullarsi (con la produzione $A \to \epsilon$). A è seguito da b (nella produzione $S \to Ab$) e quindi sceglieremo la produzione $A \to \epsilon$.
$(q, ab, S) \vdash_N (q, ab, Ab) \vdash_N (q, ab, aAb) \vdash_N (q, b, Ab) \vdash_N (q, b, b) \vdash_N (q, \epsilon, \epsilon)$

La derivazione leftmost è $S \Rightarrow Ab \Rightarrow aAb \Rightarrow ab$

Follow

Data una grammatica libera G e $A \in NT$, diciamo che $Follow(A)$ è l'insieme dei **terminali** che possono comparire immediatamente a destra di A in una forma sentenziale derivata dal simbolo iniziale S.

- per $a \in T$, $a \in Follow(A)$ sse $S \Rightarrow^* \alpha A a \beta$ per qualche $\alpha, \beta \in (T \cup NT)^*$
- inoltre, se $S \Rightarrow^* \alpha A$, allora $\$ \in Follow(A)$
  - poichè $S \Rightarrow^* S$, allora $\$ \in Follow(S)$ (per default)

Quindi: $Follow : NT \rightarrow \mathcal{P}(T \cup \{\$\})$, cioè $Follow(A) \subseteq T \cup \{\$\}$.

Riconsideriamo l'esempio:

$$S \rightarrow Ab \mid c$$

$$A \rightarrow aA \mid \epsilon$$

$Follow(S) = \{\$\}$ perché $S \Rightarrow^* S$
$Follow(A) = \{b\}$ perché $S \Rightarrow^* Ab$

Considerando i First dei corpi delle produzioni per un nonterminale A, ed anche i suoi Follow (se A è annullabile), in molti casi si potrà scegliere la produzione da espandere in modo deterministico.

Come calcolare i First?

Algoritmo standard

Sia $N(G) \subseteq NT$ l'insieme dei simboli annullabili ($A \in N(G)$ se $A \Rightarrow^+ \epsilon$).

- Per ogni $a \in T$, $First(a) = \{a\}$
- Per ogni $X \in NT$, inizializza $First(X) = \emptyset$
- Ripeti il seguente ciclo finchè nessun $First(X)$ viene più modificato in una iterazione:
  - Per ogni produzione $X \to Y_1 \ldots Y_K$
    - per ogni i da 1 a k:
      se $Y_1, \ldots, Y_{i-1} \in N(G)$ allora $First(X) := First(X) \cup (First(Y_i) \setminus \{\epsilon\})$
- Per ogni $X \in N(G)$, $First(X) := First(X) \cup \{\epsilon\}$

Come calcolare i First? (2)

Algoritmo operativo (ma può andare in loop in presenza di ricorsione sinistra $A \Rightarrow^+ A\alpha$)

Sia $N(G) \subseteq NT$ l'insieme dei simboli annullabili ($A \in N(G)$ se $A \Rightarrow^+ \epsilon$).

- Per ogni $a \in T$, $First(a) = \{a\}$
- Per ogni $A \in NT$, con produzioni $A \to \alpha_1 \mid ... \mid \alpha_k$, $First(A) = First(\alpha_1) \cup ... \cup First(\alpha_k)$

dove $First$ di $\alpha \in (T \cup NT)^*$ è definito induttivamente come segue:

- $First(\epsilon) = \{\epsilon\}$
- $First(X\beta) = \begin{cases} First(X) & \text{se } X \notin N(G) \\ (First(X) \setminus \{\epsilon\}) \cup First(\beta) & \text{se } X \in N(G) \end{cases}$

Esempio

Riprendiamo la grammatica

$$S \to Ab \mid c$$
$$A \to aA \mid \epsilon$$

Ho che

$$\begin{aligned} First(A) &= First(aA) \cup First(\epsilon) \\ &= First(a) \cup \{\epsilon\} = \{a\} \cup \{\epsilon\} = \{a, \epsilon\} \\ First(S) &= First(Ab) \cup First(c) \\ &= (First(A) \setminus \{\epsilon\}) \cup First(b) \cup \{c\} \\ &= \{a\} \cup \{b\} \cup \{c\} = \{a, b, c\} \end{aligned}$$

Esempio (2)

$$S \to ABC \mid CB$$
$$A \to a \mid \epsilon$$
$$B \to b \mid \epsilon$$
$$C \to c$$

$$First(A) = \{a, \epsilon\}$$
$$First(B) = \{b, \epsilon\}$$
$$First(C) = \{c\}$$
$$\begin{aligned} First(S) &= First(ABC) \cup First(CB) \\ &= (First(A) \setminus \{\epsilon\}) \cup First(BC) \cup First(C) \\ &= \{a\} \cup (First(B) \setminus \{\epsilon\}) \cup First(C) \cup \{c\} \\ &= \{a\} \cup \{b\} \cup \{c\} = \{a, b, c\} \end{aligned}$$

Come calcolare i Follow?

- Inizializza $\mathit{Follow}(S) = \{\$\}$.
- Per ogni altro $Y \in NT$, inizializza $\mathit{Follow}(Y) = \emptyset$.
- Ripeti il seguente ciclo finchè nessun $\mathit{Follow}(Y)$ viene più modificato in una iterazione:
  - Per ogni produzione $X \to \alpha Y \beta$:
    - $\mathit{Follow}(Y) := \mathit{Follow}(Y) \cup (\mathit{First}(\beta) \setminus \{\epsilon\})$
  - Per ogni produzione $X \to \alpha Y$ ed ogni produzione $X \to \alpha Y \beta$ con $\epsilon \in \mathit{First}(\beta)$:
    - $\mathit{Follow}(Y) := \mathit{Follow}(Y) \cup \mathit{Follow}(X)$

In pratica, bisogna cercare tutte le produzioni in cui $Y \in NT$ appare nel corpo, e, per ognuna di esse, applicare una delle due regole sopra (o anche entrambe).

Esempio

$$S \to ABC$$
$$A \to aaA|\epsilon$$
$$B \to b|\epsilon$$
$$C \to cC|\epsilon$$

	First	Follow
S	a, b, c, ϵ	\$
A	a, ϵ	$b, c, \$$
B	b, ϵ	$c, \$$
C	c, ϵ	\$

Consideriamo la produzione $S \to ABC$.

- Vale che $Follow(B) \supseteq First(C) \setminus \{\epsilon\}$ (per la prima regola).
- Inoltre, poiché $\epsilon \in First(C)$, vale che $Follow(B) \supseteq Follow(S)$ (per la seconda regola).
- Similmente, si fa il calcolo di $Follow(A)$:
  - $Follow(A) \supseteq First(BC) \setminus \{\epsilon\} = \{b, c\}$ (per la prima regola).
  - $Follow(A) \supseteq Follow(S) = \{\$\}$ (per la seconda regola).

Esempio

Grammatica non ambigua e fattorizzata per le espressioni aritmetiche:

$$E \to TE'$$
$$E' \to \epsilon \mid +E \mid -E$$
$$T \to AT'$$
$$T' \to \epsilon \mid *T$$
$$A \to a \mid b \mid (E)$$

	First	Follow
E	$a, b, ($	$\$,)$
E'	$\epsilon, +, -$	$\$,)$
T	$a, b, ($	$\$,), +, -$
T'	$\epsilon, *$	$\$,), +, -$
A	$a, b, ($	$\$,), +, -, *$

- $\$ \in \mathit{Follow}(E)$ perché E è il simbolo iniziale.
- $) \in \mathit{Follow}(E)$ per la produzione $A \to (E)$.
- $\mathit{Follow}(E) = \mathit{Follow}(E')$ perché:
  - $\mathit{Follow}(E') \subseteq \mathit{Follow}(E)$ per le produzioni $E' \to +E \mid -E$.
  - $\mathit{Follow}(E) \subseteq \mathit{Follow}(E')$ per la produzione $E \to TE'$.

Esempio – continua

$$E \to TE'$$
$$E' \to \epsilon \mid +E \mid -E$$
$$T \to AT'$$
$$T' \to \epsilon \mid *T$$
$$A \to a \mid b \mid (E)$$

	First	Follow
E	$a, b, ($	$\$,)$
E'	$\epsilon, +, -$	$\$,)$
T	$a, b, ($	$\$,), +, -$
T'	$\epsilon, *$	$\$,), +, -$
A	$a, b, ($	$\$,), +, -, *$

- Per la produzione $E \to TE'$,
  - $(First(E') \setminus \{\epsilon\}) \subseteq Follow(T)$ (prima regola)
  - poiché $\epsilon \in First(E')$, $Follow(E) \subseteq Follow(T)$ (seconda regola)
- $Follow(T) = Follow(T')$ perché:
  - $Follow(T') \subseteq Follow(T)$ per la produzione $T' \to *T$,
  - $Follow(T) \subseteq Follow(T')$ per la produzione $T \to AT'$.
- Per la produzione $T \to AT'$,
  - $(First(T') \setminus \{\epsilon\}) \subseteq Follow(A)$ (prima regola).
  - Poiché $\epsilon \in First(T')$, $Follow(T) \subseteq Follow(A)$ (seconda regola).

Parser LL(1)

Tabella di parsing LL(1)

La tabella di parsing **LL(1)**, dove:

- la prima L sta per input letto Left-to-right,
- la seconda L sta ad indicare che si costruisce una Leftmost derivation, e
- 1 sta ad indicare che si considera un solo simbolo di look-ahead,

è lo strumento che serve per risolvere il nondeterminismo.

La tabella è una matrice bidimensionale M:

- righe: nonterminali NT
- colonne: $T \cup \{\$\}$
- ogni casella $M[A, b]$ contiene le produzioni che possono essere scelte dal parser mentre tenta di espandere A (sulla cima della pila) e l'input corrente è b.

Tabella di parsing LL(1)

Come riempire la tabella di parsing LL(1)?

- Per ogni produzione $A \to \alpha$ della grammatica:
  - Per ogni $b \in T$ e $b \in \mathit{First}(\alpha)$, inserisci $A \to \alpha$ nella casella $M[A, b]$.
  - Se $\epsilon \in \mathit{First}(\alpha)$, inserisci $A \to \alpha$ in tutte le caselle $M[A, x]$ dove $x \in \mathit{Follow}(A)$.

Se ogni casella contiene al più una produzione, allora il parser è deterministico e la grammatica è detta di classe LL(1).

Se, dopo aver elaborato tutte le produzioni, una casella $M[A, b]$ risulta vuota, allora il parser, se trova A in cima alla pila e il simbolo in lettura è proprio b, si blocca e segnala l'errore!

Grammatica LL(1) e Linguaggio LL(1)

Grammatica LL(1)
Una grammatica è di classe **LL(1)** se e solo se ogni casella della tabella di parsing LL(1) contiene al più una produzione.

Linguaggio LL(1)
Un linguaggio L è di classe **LL(1)** se e solo se esiste una grammatica G di classe LL(1) tale che $L = L(G)$.

Se la grammatica è LL(1), allora il parser, sfruttando la tabella di parsing LL(1), è un parser **predittivo** deterministico: il parser ricostruisce l'albero di derivazione corrispondente alla derivazione sinistra per l'input w, in modo top-down, predicendo quale produzione usare (tra le molte possibili) guardando il prossimo carattere dell'input.

Esempio

Consideriamo la grammatica e la sua tabella dei First/Follow:

$$S \to aSb \mid \epsilon$$

	First	Follow
S	a, ϵ	$\$, b$

La tabella di parsing LL(1) è:

	a	b	$\$$
S	$S \to aSb$	$S \to \epsilon$	$S \to \epsilon$

Poiché ogni entrata della tabella contiene al più una produzione, la grammatica è di classe LL(1).

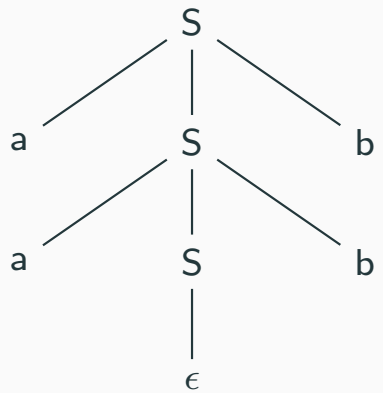
Come funziona il parser LL(1)?

Esempio – come funziona il parser LL(1)?

- Poiché il PDA ha un solo stato, non serve menzionarlo. Possiamo semplicemente considerare l'input e la pila.
- Per riconoscere se la pila è stata svuotata, aggiungiamo un simbolo \$ di fine pila, sul quale inizialmente mettiamo il simbolo iniziale S della grammatica.
- Ad ogni azione di tipo "espandi", il parser fornisce in output la corrispondente produzione, generando l'albero di derivazione in modo top-down.

input	stack
$\underline{a}abb\$$	$S\$$
	$\underline{a}Sb\$$
$\underline{a}bb\$$	$Sb\$$
	$\underline{a}Sbb\$$
$\underline{b}b\$$	$Sbb\$$
	$\underline{b}b\$$
$\underline{b}\$$	$\underline{b}\$$
$\$$	$\$$

Output:
albero di derivazione



Domanda: Perché $M[S, \$] = S \to \epsilon$? Quando serve?

Parser predittivo LL(1)

```
1:  Pila ← S$                                          ▷ cima della pila a sinistra
2:  X ← S                                              ▷ X vale il top della pila
3:  input ← w$
4:  i_c ← primo carattere dell'input
5:  while X ≠ $ do                                     ▷ finché la pila non è vuota
6:      if X terminale then
7:          if X = i_c then                            ▷ caso match, azione "consuma"
8:              pop X dalla pila
9:              avanza i_c sull'input
10:         else
11:             error                                  ▷ caso di mancato match
12:         end if
13:     else                                           ▷ X è un nonterminale
14:         if M[X, i_c] = (X → Y_1 ··· Y_n) then      ▷ azione "espandi"
15:             pop X dalla pila
16:             push Y_1 ··· Y_n sulla pila            ▷ Y_1 è il top della pila, quindi derivazione leftmost
17:             output X → Y_1 ··· Y_n
18:         else
19:             error                                  ▷ casella vuota della tabella
20:         end if
21:     end if
22:     X ← top della pila
23: end while
24: if i_c ≠ $ then
25:     error                                          ▷ pila vuota ma input non terminato
26: end if
```

Esempio: grammatica delle espressioni aritmetiche

$$E \to TE'$$

$$E' \to \epsilon \mid +E \mid -E$$

$$T \to AT'$$

$$T' \to \epsilon \mid *T$$

$$A \to a \mid b \mid (E)$$

	First	Follow
E	$a, b, ($	$\$,)$
E'	$\epsilon, +, -$	$\$,)$
T	$a, b, ($	$\$,), +, -$
T'	$\epsilon, *$	$\$,), +, -$
A	$a, b, ($	$\$,), +, -, *$

Tabella di parsing LL(1)

	a	b	$($	$)$	$+$	$-$	$*$	$\$$
E	$E \to TE'$	$E \to TE'$	$E \to TE'$					
E'				$E' \to \epsilon$	$E' \to +E$	$E' \to -E$		$E' \to \epsilon$
T	$T \to AT'$	$T \to AT'$	$T \to AT'$					
T'				$T' \to \epsilon$	$T' \to \epsilon$	$T' \to \epsilon$	$T' \to *T$	$T' \to \epsilon$
A	$A \to a$	$A \to b$	$A \to (E)$					

Esempio: grammatica delle espressioni aritmetiche

input	stack
	$E\$$
$\underline{a} * (b + a)\$$	$TE'\$$
	$AT'E'\$$
	$\underline{a}T'E'\$$
	$T'E'\$$
$\underline{*}(b + a)\$$	$\underline{*}TE'\$$
	$TE'\$$
	$AT'E'\$$
$\underline{(}b + a)\$$	$\underline{(}E)T'E'\$$
	$E)T'E'\$$
	$TE')T'E'\$$
$\underline{b} + a)\$$	$AT'E')T'E'\$$
	$\underline{b}T'E')T'E'\$$

input	stack
	$T'E')T'E'\$$
$\underline{+}a)\$$	$E')T'E'\$$
	$\underline{+}E)T'E'\$$
	$E)T'E'\$$
	$TE')T'E'\$$
$\underline{a})\$$	$AT'E')T'E'\$$
	$\underline{a}T'E')T'E'\$$
	$T'E')T'E'\$$
$\underline{)}\$$	$E')T'E'\$$
	$\underline{)}T'E'\$$
	$T'E'\$$
$\$$	$E'\$$
	$\$$

N.B.: La derivazione è proprio leftmost. **Esercizio-sfida**: costruirne l'albero.

Condizione necessaria e sufficiente per LL(1)

Teorema
G è LL(1) se e solo se per ogni coppia di produzioni distinte con la stessa testa $A \to \alpha \mid \beta$ si ha che

- $First(\alpha) \cap First(\beta) = \emptyset$
- se $\epsilon \in First(\alpha)$, allora $Follow(A) \cap First(\beta) = \emptyset$
- se $\epsilon \in First(\beta)$, allora $First(\alpha) \cap Follow(A) = \emptyset$

Dimostrazione: In base all'algoritmo di riempimento della tabella di parsing LL(1), se sono soddisfatte le due condizioni per ogni coppia di produzioni distinte con medesima testa, allora la tabella contiene al più una produzione in ogni casella.
Ma vale anche il viceversa, ovvero se la tabella di parsing non contiene più di una produzione in ciascuna casella, allora le due condizioni del teorema sono soddisfatte.

Esempio

$$S \to A \mid B$$
$$A \to ab \mid cd$$
$$B \to ad \mid cb$$

Questa G non è LL(1), perché, considerando le produzioni $S \to A \mid B$, abbiamo che $First(A) \cap First(B) = \{a, c\}$. Possiamo però manipolarla per farla diventare LL(1):

- Elimino le produzioni unitarie, ottenendo $S \to ab \mid cd \mid ad \mid cb$
- Fattorizzo e semplifico ottenendo G':

$$S \to aT \mid cT \qquad\qquad T \to b \mid d$$

che è banalmente LL(1).

N.B.: G non è LL(1), ma $L(G) = \{ab, cd, ad, cb\}$ è un linguaggio LL(1) perché esiste una grammatica LL(1) che lo genera, cioè G'!

Esempio

$S \to aAB \mid bS$

$A \to a$

$B \to b$

	First	Follow
S	a, b	\$
A	a	b
B	b	\$

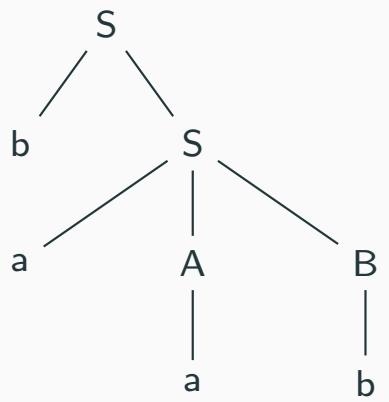
Tabella di parsing LL(1)

	a	b	\$
S	$S \to aAB$	$S \to bS$	
A	$A \to a$		
B		$B \to b$	

Esempio (2)

input	stack
$\underline{b}aab\$$	$S\$$
$\underline{a}ab\$$	$\underline{b}S\$$
$\underline{a}b\$$	$S\$$
$\underline{b}\$$	$\underline{a}AB\$$
$\$$	$AB\$$
	$\underline{a}B\$$
	$B\$$
	$\underline{b}\$$
	$\$$

Output:
albero di derivazione



Esempio (3)

Tabella di parsing LL(1)

	a	b	\$
S	$S \to aAB$	$S \to bS$	
A	$A \to a$		
B		$B \to b$	

input	stack
	$S\$$
$\underline{a}bb\$$	$\underline{a}AB\$$
	$AB\$$
$\underline{b}b\$$	

Errore: casella bianca!
$M[A, b]$ vuota!

input	stack
$\underline{a}abb\$$	$S\$$
	$\underline{a}AB\$$
$\underline{a}bb\$$	$AB\$$
	$\underline{a}B\$$
$\underline{b}b\$$	$B\$$
	$\underline{b}\$$
$\underline{b}\$$	$\$$

Errore: ho svuotato la pila, ma non ho finito l'input!

Altro esempio

Sia G la seguente grammatica:

$S \to aAB \mid B$

$A \to a$

$B \to b \mid bB$

$L(G) = (aa \mid \epsilon)b^+$
G non è LL(1) per $B \to b \mid bB$.

Fattorizziamo le produzioni per B:

$B \to bB'$

$B' \to B \mid \epsilon$

	First	Follow
S	a, b	\$
A	a	b
B	b	\$
B'	b, ϵ	\$

Tabella di parsing LL(1)

	a	b	\$
S	$S \to aAB$	$S \to B$	
A	$A \to a$		
B		$B \to bB'$	
B'		$B' \to B$	$B' \to \epsilon$

Altro esempio (2)

input	stack	input	stack
		$\underline{a}abb\$$	$S\$$
$\underline{a}a\$$	$S\$$		$\underline{a}AB\$$
	$\underline{a}AB\$$	$\underline{a}bb\$$	$AB\$$
$\underline{a}\$$	$AB\$$		$\underline{a}B\$$
	$\underline{a}B\$$	$\underline{b}b\$$	$B\$$
$\$$	$B\$$		$\underline{b}B'\$$
		$\underline{b}\$$	$B'\$$
			$B\$$
		$\$$	$\underline{b}B'\$$
			$B'\$$
			$\$$

Errore: casella bianca!

$M[B, \$]$ vuota!

Ulteriore esempio

$S \to aAB$

$A \to C \mid D$

$B \to b$

$C \to c \mid \epsilon$

$D \to d$

	First	Follow
S	a	\$
A	c, d, ϵ	b
B	b	\$
C	c, ϵ	b
D	d	b

	a	b	c	d	\$
S	$S \to aAB$				
A		$A \to C$	$A \to C$	$A \to D$	
B		$B \to b$			
C		$C \to \epsilon$	$C \to c$		
D				$D \to d$	

$M[A, b] = A \to C$ perché $\epsilon \in \mathit{First}(C)$ e $b \in \mathit{Follow}(A)$
$M[C, b] = C \to \epsilon$ perché $\epsilon \in \mathit{First}(\epsilon)$ e $b \in \mathit{Follow}(C)$

Esercizio: mostrare il funzionamento del parser per gli input *adb* e *abb*.

Teorema: regolari $\subseteq$ LL(1)

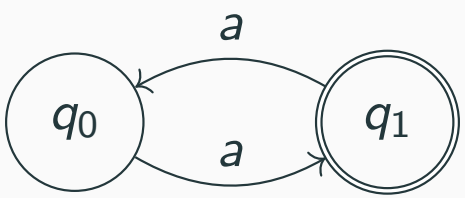
Teorema
Ogni linguaggio regolare è generabile da una grammatica G di classe LL(1). Quindi la classe dei linguaggi regolari è inclusa nella classe dei linguaggi LL(1).

Dimostrazione: Se L è regolare, allora esiste un DFA $M = (Q, \Sigma, \delta, q_0, F)$ tale che $L = L[M]$. A partire da M, costruiamo la grammatica regolare $G = (NT, T, S, R)$ con $NT = \{[q] \mid q \in Q\}$, $T = \Sigma$, $S = [q_0]$ e R definita da:

- se $\delta(q, a) = q'$ allora $[q] \to a[q'] \in R$
- se $q \in F$ allora $[q] \to \epsilon \in R$

G così definita è LL(1)! Infatti, poiché M è deterministico, da ogni $q \in Q$ per ogni $a \in \Sigma$ $\exists! q'.\ q \xrightarrow{a} q'$, cioè $[q]$ ha una sola produzione $[q] \to a[q']$ che inizia per a. Quindi le condizioni sui *First* sono soddisfatte!
Inoltre, se q è finale, allora $[q] \to \epsilon$ è applicabile solo per i $\mathit{Follow}([q]) = \{\$\}$, quindi non c'è conflitto nella tabella di parsing, perché nessuna produzione genera \$.

Esempio



$$[q_0] \rightarrow a[q_1]$$
$$[q_1] \rightarrow a[q_0] \mid \epsilon$$

La grammatica è LL(1) perché

- $First(a[q_0]) \cap First(\epsilon) = \emptyset$
- $First(a[q_0]) \cap Follow([q_1]) = \emptyset$

Parser predittivo a discesa ricorsiva

Parser predittivo a discesa ricorsiva

- Data una grammatica G di classe LL(1), è possibile costruire un parser predittivo, detto a discesa ricorsiva.
- Esso è costituito da un insieme di procedure (possibilmente mutuamente ricorsive), una per ogni nonterminale di G.
- Queste procedure condividono una variabile globale, detta i_c, che contiene il valore del prossimo carattere in lettura dell'input $w\$$.
- Si comincia invocando la procedura per il nonterminale iniziale S.
- La pila non viene gestita esplicitamente, ma implicitamente attraverso la pila dei record di attivazione delle procedure.
- La sequenza delle chiamate delle procedure determina la derivazione leftmost (se l'input è legale).

Data una grammatica libera $G = (NT, T, S, R)$ di classe LL(1) (quindi fornita di tabella di parsing M), per ogni nonterminale A con produzioni $A \to X_1^1 ... X_{n_1}^1 \mid ... \mid X_1^k ... X_{n_k}^k$ definisci la procedura $A()$ come segue:

Procedura per il nonterminale A

```
1:  procedure A()
2:  if M[A, i_c] = (A → X_1^h ··· X_{n_h}^h) then        ▷ seleziono la produzione dalla tabella
3:      output A → X_1^h ··· X_{n_h}^h                  ▷ e costruisco un passo di derivazione leftmost
4:      for i = 1 to n_h do
5:          if X_i^h ∈ NT then                           ▷ azione "espandi" chiamando la procedura X_i^h
6:              call X_i^h()
7:          else
8:              if X_i^h = i_c then                      ▷ caso match, azione "consuma"
9:                  avanza i_c sull'input
10:             else
11:                 error                                ▷ caso di mancato "match"
12:             end if
13:         end if
14:     end for
15: else
16:     error                                            ▷ caso di casella bianca
17: end if
```

Grammatiche LL(k)

$\mathbf{First}_k$ e $\mathbf{Follow}_k$

È possibile generalizzare l'approccio LL per un qualunque $k \geq 1$.
Bisogna, innanzitutto, generalizzare le funzioni *First* e *Follow*, parametrizzate da k.

$\mathbf{First}_k(\alpha)$

$w \in \mathit{First}_k(\alpha)$ se e solo se

- $\alpha \Rightarrow^* w\beta$ con $|w| = k$, $w \in T^+$, $\beta \in (T \cup NT)^*$, oppure
- $\alpha \Rightarrow^* w$ con $|w| \leq k$, $w \in T^*$ (w può essere ϵ).

$\mathbf{Follow}_k(A)$

$w \in \mathit{Follow}_k(A)$ se e solo se

- $S \Rightarrow^* \alpha A\beta$ con $w \in \mathit{First}_k(\beta) \setminus \{\epsilon\}$, oppure
- $S \Rightarrow^* \alpha A\beta$ con $\epsilon \in \mathit{First}_k(\beta)$ e $w = \$$, oppure
- $S \Rightarrow^* \alpha A$ e $w = \$$.

Look-ahead di dimensione k

Definiamo come look-ahead di dimensione k della produzione $A \to \alpha$ l'insieme

$$LA_k(A \to \alpha) = \{w \mid |w| \leq k, w \in \mathit{trunc}_k(\mathit{First}_k(\alpha) \cdot \mathit{Follow}_k(A))\}$$

cioè calcoliamo il linguaggio $\mathit{First}_k(\alpha)$ concatenato con $\mathit{Follow}_k(A)$, e poi estraiamo, troncati a lunghezza k, le parole di quel linguaggio.

Nota che $LA_1(A \to \alpha) = \begin{cases} \mathit{First}(\alpha) & \text{se } \epsilon \notin \mathit{First}(\alpha) \\ (\mathit{First}(\alpha) \setminus \{\epsilon\}) \cup \mathit{Follow}(A) & \text{altrimenti} \end{cases}$

che è esattamente l'insieme dei possibili look-ahead di dimensione 1 per $A \to \alpha$.

Tabella di parsing LL(k)

La tabella di parsing LL(k) ha:

- righe: nonterminali in NT
- colonne: $\cup_{\{A \to \alpha \in R\}} LA_k(A \to \alpha)$
  - cioè prendiamo $LA_k(A \to \alpha)$ per ogni produzione $A \to \alpha$.
- per ogni produzione $A \to \alpha$, $M[A, w]$ contiene $A \to \alpha$, per ogni $w \in LA_k(A \to \alpha)$.
  - Nota che per $k = 1$, questo è esattamente quello che si fa per riempire la tabella LL(1).

Se ogni entrata/casella contiene al più una produzione, allora G è di classe LL(k).

Teorema: G è LL(k) se per ogni coppia di produzioni con medesima testa, e.g., $A \to \alpha \mid \beta$, vale che $LA_k(A \to \alpha) \cap LA_k(A \to \beta) = \emptyset$.

Esempio

$G = S \to aSb \mid ab \mid c \qquad L(G) = \{a^n b^n | n \geq 1\} \cup \{a^n c b^n | n \geq 0\}$

- $First_2(aSb) = \{aa, ac\}$
- $First_2(ab) = \{ab\}$
- $First_2(c) = \{c\}$
- $Follow_2(S) = \{b, bb\}$

- $LA_2(S \to aSb) = trunc_2(First_2(aSb) \cdot Follow_(S)) = \{aa, ac\}$
- $LA_2(S \to ab) = trunc_2(First_2(ab) \cdot Follow_2(S)) = \{ab\}$
- $LA_2(S \to c) = trunc_2(First_2(c) \cdot Follow_2(S)) = \{cb\}$

Tabella di parsing LL(2)

	aa	ab	ac	cb
S	$S \to aSb$	$S \to ab$	$S \to aSb$	$S \to c$

Tabella di parsing LL(2) senza conflitti, quindi la grammatica è LL(2)!

Altro esempio

$S \to aSb \mid aAc$

- $First_2(aSb) = \{aa\}$
- $First_2(aAc) = \{ac\}$
- $First_2(cA) = \{cc\}$
- $First_2(c) = \{c\}$
- $Follow_2(S) = \{b, bb\}$
- $Follow_2(A) = \{c, cb\}$

$A \to cA \mid c$

- $LA_2(S \to aSb) = trunc_2(First_2(aSb) \cdot Follow_(S)) = \{aa\}$
- $LA_2(S \to aAc) = trunc_2(First_2(aAc) \cdot Follow_2(A)) = \{ac\}$
- $LA_2(A \to cA) = trunc_2(First_2(cA) \cdot Follow_2(A)) = \{cc\}$
- $LA_2(A \to c) = trunc_2(First_2(c) \cdot Follow_2(A)) = \{cc\}$

	aa	ac	cc
S	$S \to aSb$	$S \to aAc$	
A			$A \to cA \quad A \to c$

Tabella di parsing LL(2) con conflitti, quindi la grammatica non è di classe LL(2).

Esercizi

- Considera la grammatica G che modifica leggermente quella precedente:

$$S \to aSb \mid aAc \qquad\qquad A \to cA \mid d$$

  Dimostra che G è di classe LL(2).

- Si consideri la grammatica G con simbolo iniziale S:

$$S \quad \to \quad \mathtt{abc}S \mid \mathtt{abd}S \mid \epsilon$$

  (*i*) Dimostrare che G è di classe LL(3). (*ii*) Quale linguaggio genera G? (*iii*) Manipolare G per ottenerne una equivalente di classe LL(1).

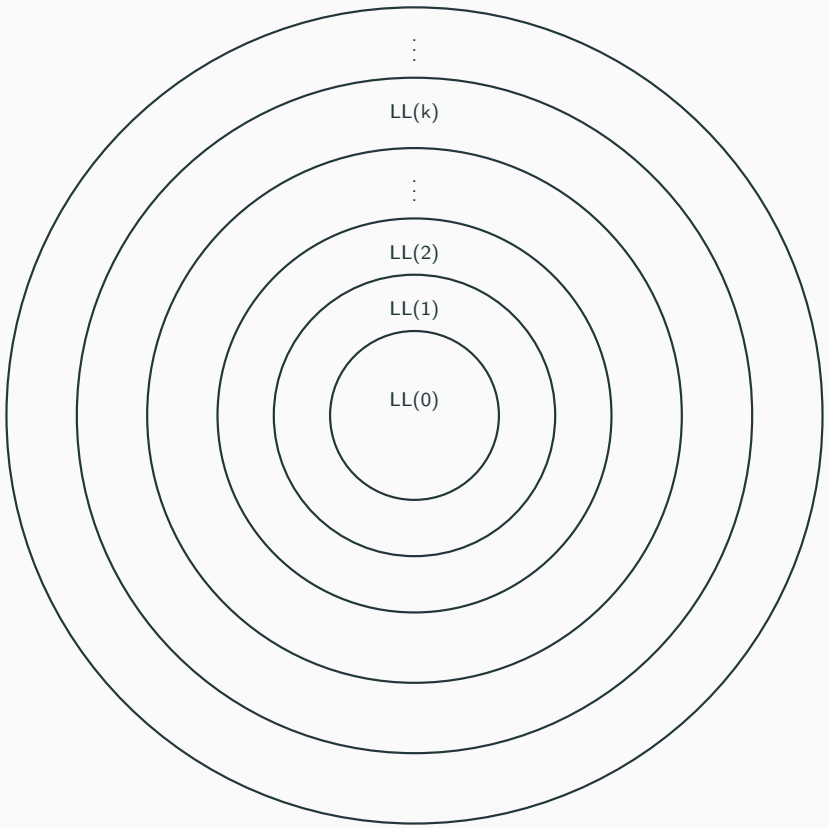
Teoremi su grammatiche LL(K)

Teorema

- Una grammatica ricorsiva sinistra non è LL(k) per nessun k. (Perché?)
- Una grammatica ambigua non è LL(k) per nessun k.
- Se G è LL(k) per qualche k, allora G non è ambigua.
- Se G è LL(k), allora $L(G)$ è libero deterministico.
- Esiste L libero deterministico tale che non esiste G di classe LL(k) - per nessun k- tale che $L = L(G)$.

Linguaggi LL(K)

Un linguaggio L è di classe LL(k) se esiste una grammatica G di classe LL(k) tale che $L = L(G)$.
Per ogni $k \geq 0$, la classe di linguaggi LL(k+1) contiene strettamente la classe dei linguaggi LL(k).



N.B: G è LL(0) se ogni $A \in NT$ ha una sola produzione, quindi $L(G)$ contiene al più una parola. Quindi, LL(0) è inutile!

Linguaggi LL(K)

Nella pratica si usa solo LL(1). Se G non è LL(1), spesso la si può manipolare (fattorizzazione, eliminare ricorsione sinistra, disambiguare, ecc...) trasformandola in una equivalente LL(1). Ad esempio:

$G = S \to aSb \mid ab \mid c$ è LL(2) (visto in precedenza!) ma non LL(1).

$G' = \begin{cases} S \to aT \mid c \\ T \to Sb \mid b \end{cases}$ ottenuta dalla fattorizzazione di G, è LL(1)!

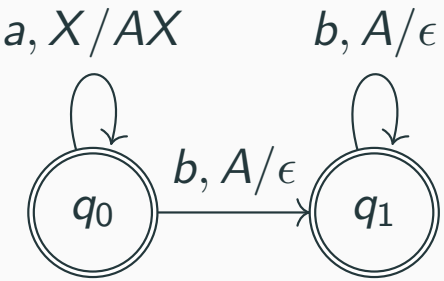
Infatti

- $First(aT) \cap First(c) = \emptyset$
- $First(Sb) \cap First(b) = \emptyset$

$L = \{a^n b^n | n \geq 1\} \cup \{a^n c b^n | n \geq 0\}$ è un linguaggio di classe LL(1) perché G' è una grammatica LL(1)!

LL(k) $\subset$ Liberi deterministici

$L = \{a^i b^j | i \geq j\}$ è libero deterministico, ma non è LL(k) per nessun k!



Questo DPDA riconosce L per stato finale.

Possibile grammatica libera per L: $G = \begin{cases} S \to aS \mid B \\ B \to aBb \mid \epsilon \end{cases}$

Come faccio a scegliere tra $S \to aS$ e $S \to B$? Dovrei leggere fino in fondo all'input per sapere quante b in meno di a ci sono nella stringa, quindi G non può essere LL(k) per nessun k.
Si può dimostrare che non è possibile trovare G' e k tali che $L(G') = L$ e $G' \in LL(k)$.

Esercizio: un linguaggio LL(1)

Nota che il linguaggio $L = \{a^i b^j | j \geq i \geq 0\}$, molto simile al precedente, è invece un linguaggio di classe LL(1).

Definiamo G tale che $L(G) = L$ come $G = \begin{cases} S \to S_1 B \\ S_1 \to aS_1 b \mid \epsilon \\ B \to bB \mid \epsilon \end{cases}$

	First	Follow
S	a, b, ϵ	\$
S_1	a, ϵ	b, \$
B	b, ϵ	\$

Tabella di parsing LL(1)

	a	b	\$
S	$S \to S_1 B$	$S \to S_1 B$	$S \to S_1 B$
S_1	$S_1 \to aS_1 b$	$S_1 \to \epsilon$	$S_1 \to \epsilon$
B		$B \to bB$	$B \to \epsilon$

Separazione tra LL(k) e LL(k-1)

Per ogni $k \geq 2$, definiamo la grammatica

$$G_k = \begin{cases} S \to aSA \mid \epsilon \\ A \to a^{k-1}bS \mid c \end{cases}$$

Si può dimostrare che $L(G_k)$ è un linguaggio LL(k) per il quale non esiste una grammatica LL(k-1) che lo generi.
Ad esempio:

$$G_2 = \begin{cases} S \to aSA \mid \epsilon \\ A \to abS \mid c \end{cases}$$

G_2 non è LL(1) per via di $S \to aSA \mid \epsilon$, ma è LL(2) (prova a verificarlo!), e si può dimostrare che non esiste G' di classe LL(1) che genera $L(G_2)$!